HW1 1D Gaussian Filtering

20211402 김세준

1. 디바이스 정보

```
 CUDA Device Query (Runtime API) version (CUDART static linking)

Detected 1 CUDA Capable device(s)

Device 0: "NVIDIA GeForce RTX 4080 SUPER"
  CUDA Driver Version / Runtime Version          13.2 / 13.2
  CUDA Capability Major/Minor version number:    8.9
  Total amount of global memory:                 16376 MBytes (17170956288 bytes)
  (080) Multiprocessors, (128) CUDA Cores/MP:    10240 CUDA Cores
  GPU Max Clock rate:                            2595 MHz (2.60 GHz)
  Memory Clock rate:                             11501 Mhz
  Memory Bus Width:                              256-bit
  L2 Cache Size:                                 67108864 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(131072), 2D=(131072, 65536), 3D=(16384, 16384, 16384)
  Maximum Layered 1D Texture Size, (num) layers  1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(32768, 32768), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total shared memory per multiprocessor:        102400 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  1536
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:          Yes with 1 copy engine(s)
  Run time limit on kernels:                     Yes
  Integrated GPU sharing Host Memory:            No
  Support host page-locked memory mapping:       Yes
  Alignment requirement for Surfaces:            Yes
  Device has ECC support:                        Disabled
  CUDA Device Driver Mode (TCC or WDDM):         WDDM (Windows Display Driver Model)
  Device supports Unified Addressing (UVA):      Yes
  Device supports Managed Memory:                Yes
  Device supports Compute Preemption:            Yes
  Supports Cooperative Kernel Launch:            Yes
  Device PCI Domain ID / Bus ID / location ID:   0 / 1 / 0
  Compute Mode:
     < Default (multiple host threads can use ::cudaSetDevice() with device simultaneously) >

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 13.2, CUDA Runtime Version = 13.2, NumDevs = 1
Result = PASS
```

그림 1: nvidia-samples

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Tue_Dec_16_19:27:18_Pacific_Standard_Time_2025
Cuda compilation tools, release 13.1, V13.1.115
Build cuda_13.1.r13.1/compiler.37061995_0
```

그림 2: nvcc --version

Compute Compability 8.9, CUDA Toolkit 13.1.1 설치됨

2. 실험

모든 실험은 NUMBER_OF_TESTS_ON_GPU = 10인 상태에서 진행되었다. 즉, 모든 device에서의 측정 시간은 10회 측정의 평균이다.

2-1. Host vs Device

Default 상태에서 수행한 시간 비교다.

Filter Width / Time	Host [ms]	Device [ms]	Speedup
1	86.884	0.939	92.50
3	111.016	0.904	122.80
5	165.386	0.907	182.32
7	215.766	0.956	225.68
9	274.176	0.871	314.73
11	320.073	0.871	367.58

표 1: NUMBER_OF_ELEMENTS (1 << 26), THREADS_PER_BLOCK 256, GRID_REDUCTION_FACTOR 1

필터 너비가 커질수록 부하가 늘어나고, 그에 따라 호스트에서 걸린 시간은 확연하게 증가하는데에 비해 디바이스에서는 큰 차이가 나지 않았다. 이에 따라 확실히 디바이스에서 연산하는 것이 유리하다는 것을 확인할 수 있다.

2-2. Data size and threads per block

이제 데이터 크기와 threads per block의 영향을 확인하기 위해 filter width는 11로 고정하고 진행한다. 2-1에서 device의 성능이 host보다 압도적임을 확인했으므로 이제 device의 성능을 확인하기 위해 이제부터 host 시간은 측정하지 않는다. 여전히 grid-stride loop는 적용하지 않는다.

표에 출력한 숫자는 (측정 시간[ms]/Achieved Occupancy[%])이다. Achieved occupancy는 Nsight Compute를 통해 얻었다.

Data Size / Block Dim	128	256	1024
1 << 22	0.037/81.39	0.037/82.48	0.047/60.93
1 << 26	0.950/84.55	0.952/82.59	0.988/58.62
1 << 30	14.620/84.56	14.587/79.07	14.953/58.72

표 2: kernel_width 11, GRID_REDUCTION_FACTOR 1

Block dimension이 128, 256일 때와 1024일 때 비교해보면 1024일 때 실행 속도와 occupancy가 현저히 떨어지는 모습을 볼 수 있다. CC 8.9 기준 maximum number of resident threads per SM은 1536개인데, block당 threads가 256과 1024일 경우 SM당 block의 수는 각각 6개, 1개씩이다. 1024일 때는 결국 thread 512개어치 남는 공간이 생기는데, 이론치만 봐도 block dimension이 256일 때는 손해가 없지만, 1024일 때는 1/3의 손해를 보고 들어간다. 반면 block dimension이 128이나 256일 때는 큰 차이가 없다.

연산 결과를 block dimension 256 기준으로 data size에 대해 비교하면, 데이터 크기가 작을 때는 그 크기가 16배 증가했을 때 시간이 25.73배 증가했지만, 데이터 크기가 충분히 커졌을 때 크기가 16배 증가했을 때 시간이 15.32배 증가했다. 이는 GPU의 모든 warp가 쉬지 않는 상태에서 최대 성능을 발휘한다는 것을 보여준다.

2-3. Data size and grid-stride loop

이제는 데이터 크기와 grid-stride loop을 이용해 block 개수에 변화를 주면 어떻게 될지 확인해볼 것이다. 데이터 크기는 2-2의 것과 동일하게 변화를 주고, block dimension을 256으로 고정하고 GRID_REDUCTION_FACTOR에 변화를 줄 것이다.

표에 출력한 숫자는 (측정 시간[ms]/SM Throughput[%])이다. SM Throughput은 Nsight Compute를 통해 얻었다.

Data size/ Reduction factor	1	2	4	8	16
1 << 22	0.038/69.17	0.036/68.73	0.035/63.98	0.036/65.51	0.036/62.68
1 << 26	0.953/58.56	0.924/46.37	0.973/52.99	1.028/45.76	0.909/50.48
1 << 30	14.769/53.44	14.446/50.84	14.351/49.11	14.550/48.39	14.518/46.68

표 3: kernel_width 11, THREADS_PER_BLOCK 256

이 표에서는 (Achieved Occupancy[%])이다. 가로와 세로축은 위와 동일하다.

Data size/ Reduction factor	1	2	4	8	16
1 << 22	82.61	87.29	88.77	86.85	80.80
1 << 26	83.81	91.42	88.60	91.45	94.03
1 << 30	79.63	84.86	86.85	89.58	93.02

표 4: kernel_width 11, THREADS_PER_BLOCK 256

모든 실험에서 데이터 크기가 동일할 때는 reduction factor가 어떻든 실행 시간에는 큰 변화가 없는 모습이다. 하지만 실행 시간과 다르게 reduction factor가 커질수록, 즉 block 개수가 줄어들수록 SM throughput은 감소하지만 warp occupancy가 증가하는 모습이 보인다. SM throughput의 감소는 block을 교체하는 과정에서 필요한 instruction issuing 줄어 자연히 감소한 것으로 보인다. 그리고 warp occupancy의 증가는 block의 개수가 줄어들음으로 인해 block의 lifetime이 늘어나서, block을 교체할 때 나타나는 overhead가 사라짐으로 인해 나타나는 현상일 것으로 보인다. 이 실험을 통해서는 block의 수를 줄이고 grid-stride loop을 적용하면 실행 시간 관점의 손해를 보지 않으면서 warp occupancy는 늘릴 수 있다는 것을 확인했다.

3. 스크린샷

```
// Check difference between calculations in host and device, considering floating point errors
bool verify_gaussian(const int* image1, const int* image2, int N) {
    for (int i = 0; i < N; i++) {
        if (abs(image1[i] - image2[i]) > 1) {
            fprintf(stderr, "***Mismatch detected at index %d.\nHost=%d, Device=%d***\n");
            return false;
        }
    }
    return true;
}
```

그림 3: verify_gaussian function

```
********************************************************************************
Experiment 1: Change filter width and compare host and device
Control Variable: NUMBER_OF_ELEMENTS = (1 << 26), GRID_REDUCTION_FACTOR = 1
Manipulated: kernel_width = (1, 3, 5, 7, 9, 11)
********************************************************************************

[1D Gaussian filtering of image of 67108864 pixels(KERNEL WIDTH = 1)]
^^^ Time to filter an image of 67108864 pixels on the host = 89.134(ms)

^^^ CUDA kernel launch with 262144(262144) blocks of 256 threads
^^^ Time to filter an image of 67108864 pixels on the device = 0.903(ms)
^^^ Speedup: 98.67x
^^^ Test PASSED

[1D Gaussian filtering of image of 67108864 pixels(KERNEL WIDTH = 3)]
^^^ Time to filter an image of 67108864 pixels on the host = 110.433(ms)

^^^ CUDA kernel launch with 262144(262144) blocks of 256 threads
^^^ Time to filter an image of 67108864 pixels on the device = 0.875(ms)
^^^ Speedup: 126.21x
^^^ Test PASSED

[1D Gaussian filtering of image of 67108864 pixels(KERNEL WIDTH = 5)]
^^^ Time to filter an image of 67108864 pixels on the host = 166.135(ms)

^^^ CUDA kernel launch with 262144(262144) blocks of 256 threads
^^^ Time to filter an image of 67108864 pixels on the device = 0.883(ms)
^^^ Speedup: 188.10x
^^^ Test PASSED
```

```
********************************************************************************
Experiment 2: Change data size and threads per block (no grid-stride loop yet)
Control Variable: kernel_width = 11, GRID_REDUCTION_FACTOR = 1
Manipulated: NUMBER_OF_ELEMENTS = (1 << 22, 1 << 26, 1 << 30)
             THREADS_PER_BLOCK = (128, 256, 1024)
********************************************************************************

Running host code for verification...
Finished!!

^^^ Total data size: 4194304, threads per block: 128
^^^ CUDA kernel launch with 32768(32768) blocks of 128 threads
^^^ Time to filter an image of 4194304 pixels on the device = 0.035(ms)
^^^ Test PASSED

^^^ Total data size: 4194304, threads per block: 256
^^^ CUDA kernel launch with 16384(16384) blocks of 256 threads
^^^ Time to filter an image of 4194304 pixels on the device = 0.054(ms)
^^^ Test PASSED

^^^ Total data size: 4194304, threads per block: 1024
^^^ CUDA kernel launch with 4096(4096) blocks of 1024 threads
^^^ Time to filter an image of 4194304 pixels on the device = 0.044(ms)
^^^ Test PASSED

Running host code for verification...
Finished!!

^^^ Total data size: 67108864, threads per block: 128
^^^ CUDA kernel launch with 524288(524288) blocks of 128 threads
```

```
********************************************************************************
Experiment 3: Change data size and test grid-stride loop
Control Variable: kernel_width = 11, THREADS_PER_BLOCK = 256
Manipulated: NUMBER_OF_ELEMENTS = (1 << 22, 1 << 26, 1 << 30)
             GRID_REDUCTION_FACTOR = (1, 2, 4, 8, 16)
********************************************************************************

Running host code for verification...
Finished!!

^^^ Total data size: 4194304, grid reduction factor: 1
^^^ CUDA kernel launch with 16384(16384) blocks of 256 threads
^^^ Time to filter an image of 4194304 pixels on the device = 0.035(ms)
^^^ Test PASSED

^^^ Total data size: 4194304, grid reduction factor: 2
^^^ CUDA kernel launch with 8192(16384) blocks of 256 threads
^^^ Time to filter an image of 4194304 pixels on the device = 0.033(ms)
^^^ Test PASSED

^^^ Total data size: 4194304, grid reduction factor: 4
^^^ CUDA kernel launch with 4096(16384) blocks of 256 threads
^^^ Time to filter an image of 4194304 pixels on the device = 0.033(ms)
^^^ Test PASSED

^^^ Total data size: 4194304, grid reduction factor: 8
^^^ CUDA kernel launch with 2048(16384) blocks of 256 threads
^^^ Time to filter an image of 4194304 pixels on the device = 0.072(ms)
^^^ Test PASSED
```

```
^^^ Total data size: 1073741824, grid reduction factor: 2
^^^ CUDA kernel launch with 2097152(4194304) blocks of 256 threads
^^^ Time to filter an image of 1073741824 pixels on the device = 13.935(ms)
^^^ Test PASSED

^^^ Total data size: 1073741824, grid reduction factor: 4
^^^ CUDA kernel launch with 1048576(4194304) blocks of 256 threads
^^^ Time to filter an image of 1073741824 pixels on the device = 13.540(ms)
^^^ Test PASSED

^^^ Total data size: 1073741824, grid reduction factor: 8
^^^ CUDA kernel launch with 524288(4194304) blocks of 256 threads
^^^ Time to filter an image of 1073741824 pixels on the device = 13.664(ms)
^^^ Test PASSED

^^^ Total data size: 1073741824, grid reduction factor: 16
^^^ CUDA kernel launch with 262144(4194304) blocks of 256 threads
^^^ Time to filter an image of 1073741824 pixels on the device = 13.767(ms)
^^^ Test PASSED


^^^ Total number of tests: 30
^^^ Passed: 30, Failed: 0

^^^ Done

C                                                  \x64\Release\Gaussian_Filtering_1D.exe(프로세스 1912)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```